



Amrita School of Computing

Amritapuri Campus

Department of Computer Science and Engineering

B.Tech Computer Science and Engineering

23CSE498 – Project Phase II

Design and Solution Submission

Project Title An Agentic AI-Powered Orthopedic Postoperative Follow-up System with Multi-Modal Multi-Agent Collaboration and Offline Deployment

Project Guide Ms. Gopika Muraleedas

Team Members

Team No.	Roll Number	Name of Student
B7	AM.SC.U4CSE23147	Rishi Priyan V N
B7	AM.SC.U4CSE23159	Sujith Matam
B7	AM.SC.U4CSE23172	Pranav P Nair

Academic Year: 2026–2027

Amrita Vishwa Vidyapeetham

1 Sustainable Development Goal (SDG) Mapping

Primary SDG: SDG 3 – Good Health and Well-being

Secondary SDG(s): SDG 9 – Industry, Innovation and Infrastructure; SDG 10 – Reduced Inequalities

Justification:

This project directly addresses **SDG 3 (Good Health and Well-being)** by deploying an AI-powered postoperative care assistant that provides continuous, personalized recovery guidance to below-hip orthopedic patients after discharge. By monitoring pain trends, medication adherence, rehabilitation progress, and psychological wellbeing, the system reduces complications and improves health outcomes without requiring costly in-person follow-up visits. It supports **SDG 9** by leveraging cutting-edge offline AI technologies (quantized Small Language Models, on-device RAG pipelines) that push the frontier of healthcare innovation. Additionally, it advances **SDG 10** by extending quality postoperative care access to patients in resource-limited or remote environments who cannot easily revisit specialist clinics, reducing inequalities in orthopedic healthcare delivery.

2 Project Milestones and Timeline

Semester	Major Activity / Milestone	Expected Deliverable	Expected Completion	Status
S6	Project domain exploration	Problem domain identified	15 Feb 2026	Completed
S6	Literature review	Initial literature survey	15 Mar 2026	Completed
S6	Baseline implementation	Reference implementation completed	30 Apr 2026	Completed
S7 (Submission 1)	Literature survey	Comprehensive literature survey	15 Jul 2026	Completed
S7 (Submission 1)	Research gap identification	Research gap statement	20 Jul 2026	Completed
S7 (Submission 1)	Problem statement	Finalized objectives	25 Jul 2026	Completed
S7 (Submission 2)	System design	Architecture and workflow	06 Aug 2026	Current
S7	Prototype implementation	Functional prototype	15 Oct 2026	Planned
S7	Module integration	Integrated system	15 Nov 2026	Planned
S7	Testing and validation	Experimental results	15 Dec 2026	Planned
S8	System enhancement	Improved system	31 Jan 2027	Planned
S8	Performance evaluation	Performance analysis	28 Feb 2027	Planned
S8	Research publication	Paper submission	20 Mar 2027	Planned
S8	Dissertation writing	Final report	10 Apr 2027	Planned
S8	Final demonstration	Presentation and viva	20 Apr 2027	Planned

Table 1: Overall Project Timeline

3 System Architecture

The proposed platform is a privacy-preserving, offline-first multi-agent ecosystem specifically engineered for below-hip orthopedic postoperative follow-up care (Hip, Knee, Lower Leg, Ankle & Foot). The system integrates four primary layers: a Mobile Patient Application serving as the presentation and capture interface, an On-Device Local Agentic Orchestrator providing safety triage and intent routing, a Specialized Multi-Agent Execution Pool of 11 domain-specific agents, and an Offline AI Engine leveraging a quantized Large Action Model (LAM) with a Retrieval-Augmented Generation (RAG) knowledge pipeline. An automated clinician reporting pipeline completes the ecosystem by generating structured weekly physician summaries.

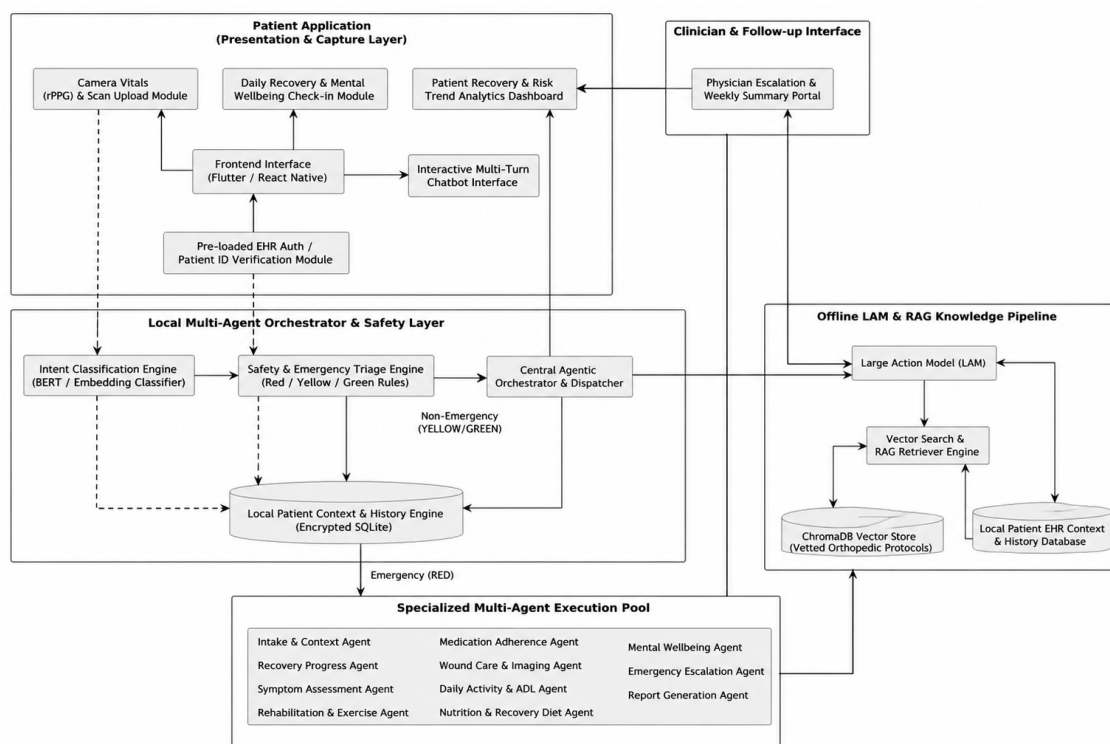


Figure 1: System Architecture of the Proposed Agentic AI-Powered Orthopedic Postoperative Follow-up System

Architecture Description

The architecture is organized into five distinct layers communicating via well-defined interfaces:

- **Mobile Patient Application (Presentation & Capture Layer):** The Flutter-based mobile frontend encompasses pre-loaded patient authentication, daily recovery

ery check-in forms, the interactive chatbot interface, a recovery analytics dashboard, and a camera module for rPPG vital-sign extraction and wound photo uploads.

- **Local Multi-Agent Orchestrator & Safety Layer:** The Central Agentic Orchestrator receives classified intents from the Intent Classification Engine and dispatches tasks to specialized agents. All queries first pass through the deterministic Emergency & Safety Triage Engine (Red/Yellow/Green). A local encrypted SQLite cache maintains the patient context and interaction history.
- **Specialized Multi-Agent Execution Pool:** Eleven domain agents (Intake, Recovery Progress, Symptom Assessment, Rehabilitation, Medication Adherence, Wound Care, Daily Activity, Nutrition, Mental Wellbeing, Emergency Escalation, and Report Generation) independently handle their clinical domains.
- **Offline AI Engine & RAG Knowledge Pipeline:** The Large Action Model (LAM) operates using a quantized local language model in GGUF format through `llama.cpp`. ChromaDB serves as the embedded vector store containing vetted orthopedic clinical protocols for context-grounded retrieval. The LAM uses the retrieved knowledge to select appropriate agents, tools, and postoperative care actions, while all inference and processing are performed locally without external API dependency.
- **Clinician & Follow-up Interface:** A physician escalation portal receives automated urgent alerts and structured weekly summary PDF reports from the Report Generation Agent.

Architectural Traceability to Project Objectives

Component	Functionality	Traceable Objective
Specialized Multi-Agent Pool	11 dedicated domain agents for triage, rehab, wound care, medication, mental health, and reporting.	Objective 1: Collaborative multi-agent architecture with specialized roles.
Large Action Model (LAM)	Local action-oriented model responsible for understanding patient requests, selecting appropriate agents/tools, and executing predefined postoperative care workflows without external API dependency.	Objective 2: Resource-efficient, offline-capable action model ensuring privacy-preserving patient interaction.
rPPG Camera Module + Psychometric Engine	Smartphone camera vitals (rPPG pulse/HRV) and GAD-7 / PHQ-9 / PCS mental health assessment.	Objective 3: Integrate vital-sign and psychometric tracking for anomaly detection.
Local RAG + Safety Triage Layer	ChromaDB vector search + deterministic Red-Flag Triage Engine matching clinical guidelines.	Objectives 4 & 5: Privacy-preserving, sensor-integrated, orthopedic-specialized care.

Table 2: Component-to-Objective Traceability Matrix

4 Module Description

The system comprises 14 distinct functional modules ensuring modularity, clinical safety, and strict domain specialization. Each module has clearly defined inputs, outputs, and responsibilities.

2.1 Pre-Loaded EHR Authentication & Patient Context Module

- **Purpose:** Authenticates patients using pre-assigned Hospital Patient IDs / Medical Record Numbers (MRN) without requiring manual registration, as postoperative patients are pre-enrolled during hospital discharge.
- **Inputs:** Pre-assigned Hospital Patient ID, Device Auth Access Key.

- **Outputs:** Verified patient context token, baseline surgical record session state.
- **Functionality:** Loads pre-configured surgical metadata from local encrypted storage – procedure type (e.g., Total Knee Arthroplasty), affected limb (Left/Right), surgery date, surgeon restrictions, and initial discharge prescriptions.

2.2 Intent Classification Engine

- **Purpose:** Categorizes incoming patient text or voice queries into standardized clinical intent domains in real time.
- **Inputs:** Raw user query string, prior multi-turn dialogue context.
- **Outputs:** Primary intent label (Recovery Progress, Pain/Symptoms, Rehab Exercises, Medication, Wound Care, Daily Activities, Diet, Mental Wellbeing, Emergency).
- **Functionality:** Executes a lightweight local embedding similarity / fine-tuned classifier to match user intent against pre-defined clinical intent schemas.

2.3 Emergency & Safety Triage Engine

- **Purpose:** Acts as a deterministic safety guardrail, inspecting every query for physical or psychological red flags prior to response generation.
- **Inputs:** User query string, intent label, recent pain score trends, rPPG vitals, and wound camera inputs.
- **Outputs:** Risk Classification – GREEN (Routine guidance), YELLOW (Doctor consultation recommended), RED (Emergency medical escalation).
- **Functionality:** Scans for high-risk clinical keywords indicating Deep Vein Thrombosis (DVT), Pulmonary Embolism (PE), Surgical Site Infection (SSI), joint dislocation, or acute psychological crisis.

2.4 Intake & Context Agent

- **Purpose:** Consolidates the patient's postoperative information, recent interactions, and relevant clinical context before a task is processed by a specialized agent.
- **Inputs:** Patient profile, surgical procedure, postoperative day, recovery history, previous interactions, and relevant patient context.

- **Outputs:** Structured patient-specific context supplied to the appropriate specialized agent.
- **Functionality:** Organizes the relevant patient information required for downstream agent processing and ensures that responses are generated using the appropriate postoperative context.

2.5 Recovery Progress Agent

- **Purpose:** Evaluates postoperative recovery milestones against standard clinical recovery timelines.
- **Inputs:** Postoperative day count, surgical procedure type, recovery history, and patient recovery query.
- **Outputs:** Stage-specific recovery guidance, expected milestone updates, and timeline-based reassurance.
- **Functionality:** Compares the patient's reported progress with appropriate below-hip orthopedic recovery benchmarks while considering the patient's surgical procedure and postoperative stage.

2.6 Symptom Assessment Agent

- **Purpose:** Evaluates self-reported physical symptoms including pain intensity, localized swelling, and joint stiffness.
- **Inputs:** Pain score (NPRS 0–10), pain characteristics, swelling observations or images, body temperature, and relevant patient history.
- **Outputs:** Symptom assessment, appropriate non-pharmacological guidance, and risk-related observations.
- **Functionality:** Maintains a historical pain and symptom log in the local patient context database to identify abnormal pain changes or persistent symptoms that may require clinical attention.

2.7 Rehabilitation & Exercise Agent

- **Purpose:** Provides phase-appropriate rehabilitation exercises and joint movement guidance based on the patient's postoperative condition.

- **Inputs:** Prescribed weight-bearing status (NWB, PWB, WBAT, FWB), current Range of Motion (ROM), postoperative stage, exercise history, and patient query.
- **Outputs:** Step-by-step exercise instructions, repetition targets, movement restrictions, and safety warnings.
- **Functionality:** Adjusts rehabilitation guidance according to the patient's surgical restrictions and reported recovery status, avoiding exercise advancement when safety concerns are identified.

2.8 Medication Adherence Agent

- **Purpose:** Supports adherence to prescribed postoperative medication schedules and addresses medication-related patient queries.
- **Inputs:** Prescribed medication list, dosage schedule, medication intake records, and patient medication queries.
- **Outputs:** Medication reminders, adherence information, and safety-oriented medication guidance.
- **Functionality:** Tracks medication schedules and recorded intake to help patients follow their prescribed regimen and identify potential adherence issues requiring clinical attention.

2.9 Wound Care & Imaging Agent

- **Purpose:** Provides guidance related to postoperative incision care and processes relevant smartphone camera wound captures.
- **Inputs:** Patient wound-care queries, postoperative incision images, wound observations, and relevant patient context.
- **Outputs:** Wound-care guidance, dressing-related instructions, and warnings for potentially abnormal wound characteristics.
- **Functionality:** Assesses submitted wound images and patient-reported observations for potentially concerning characteristics such as excessive redness, swelling, wound separation, or abnormal drainage.

2.10 Daily Activity & ADL Agent

- **Purpose:** Provides guidance for safely performing Activities of Daily Living (ADL) during postoperative recovery.
- **Inputs:** Patient activity query, home environment information, assistive device usage, surgical procedure, and postoperative restrictions.
- **Outputs:** Guidance for activities such as walking, stair navigation, sleeping positions, transfers, and other routine activities.
- **Functionality:** Applies procedure-specific precautions and postoperative restrictions when generating recommendations for daily activities.

2.11 Nutrition & Recovery Diet Agent

- **Purpose:** Provides dietary guidance that supports postoperative tissue recovery, nutritional adequacy, and general recovery.
- **Inputs:** Dietary preferences, surgery type, postoperative stage, dietary concerns, and patient-reported gastrointestinal symptoms.
- **Outputs:** General nutrition recommendations, protein guidance, hydration guidance, and relevant dietary considerations.
- **Functionality:** Generates recovery-oriented dietary recommendations using the patient's postoperative context and retrieved clinical knowledge.

2.12 Mental Wellbeing Agent

- **Purpose:** Supports monitoring of postoperative mental wellbeing, anxiety, mood, and fear of movement.
- **Inputs:** Mental wellbeing check-in responses, mood logs, relevant questionnaire scores, and patient conversations.
- **Outputs:** Supportive responses, wellbeing suggestions, coping strategies, and escalation indicators when significant distress is identified.
- **Functionality:** Monitors changes in reported mental wellbeing and identifies patterns that may require additional support or clinical attention.

2.13 Emergency Escalation Agent

- **Purpose:** Handles cases classified as high-risk by the Emergency & Safety Triage Engine and initiates the appropriate escalation workflow.
- **Inputs:** RED risk classification, detected warning signs, patient context, and relevant interaction history.
- **Outputs:** Immediate escalation guidance for the patient and relevant emergency information for the clinician interface.
- **Functionality:** Records the emergency event, provides appropriate immediate guidance, and forwards the relevant patient information to the Clinician & Follow-up Interface for physician attention.

2.14 Report Generation Agent

- **Purpose:** Synthesizes patient interactions, recovery trends, check-in information, and relevant clinical indicators into structured summaries.
- **Inputs:** Local interaction logs, daily check-in history, recovery trends, triage classifications, symptom records, exercise adherence data, and vital measurements.
- **Outputs:** Structured daily and weekly summaries for the Patient Recovery & Risk Trend Analytics Dashboard and the Physician Escalation & Weekly Summary Portal.
- **Functionality:** Aggregates relevant patient information and generates structured reports highlighting recovery trajectory, adherence patterns, and flagged risks for patient monitoring and clinician follow-up.

5 System Workflow

Since postoperative patients are pre-enrolled upon hospital discharge, manual user registration is bypassed entirely. Authentication occurs seamlessly via pre-loaded Hospital Patient IDs. The overall system workflow proceeds through seven primary stages from app launch to clinical reporting.

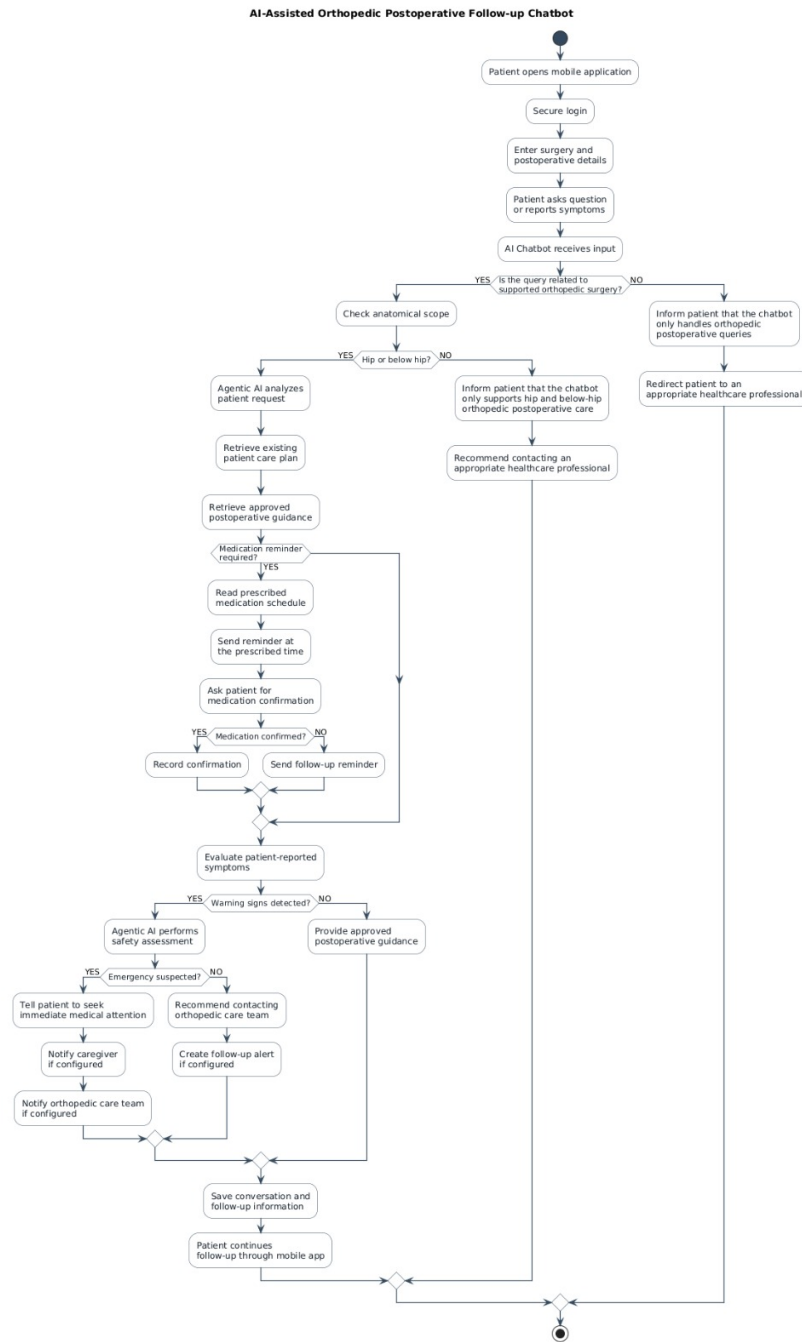


Figure 2: System Workflow of the Proposed Agentic AI-Powered Orthopedic Postoperative Follow-up System

Step-by-Step Stage Explanation

1. **Patient Authentication & Context Loading:** The patient opens the mobile application and is authenticated using the pre-loaded Hospital Patient ID. The Frontend Interface provides access to the patient's postoperative follow-up functions and associated patient context maintained within the local system.

2. **Daily Check-In & Trend Logging:** The patient completes the daily recovery and mental wellbeing check-in, recording relevant recovery indicators such as pain, swelling, wound status, exercise compliance, and wellbeing measures. The collected information is forwarded to the Local Patient Context & History Engine for secure local storage and subsequent processing.
3. **Query Submission & Intent Classification:** When a patient submits a question (e.g., *“When can I bend my knee to 90 degrees?”*), the Intent Classification Engine identifies the query category (Rehabilitation & Mobility) using cosine similarity over domain intent embeddings.
4. **Emergency Triage Engine Scan:** Every query undergoes deterministic rule-based and semantic pattern checks for red flags (e.g., calf pain + heat indicating DVT, high fever, wound pus indicating SSI). If an emergency is flagged (RED), immediate medical escalation instructions are displayed and the physician is alerted.
5. **Context Retrieval & Specialized Agent Routing:** For non-emergency queries, the Central Agentic Orchestrator coordinates the request and accesses the Local Patient Context & History Engine. Based on the identified intent and available patient context, the request is directed to the appropriate specialized agent within the Specialized Multi-Agent Execution Pool.
6. **RAG Knowledge Retrieval & LAM-Based Action Generation:** The Central Agentic Orchestrator routes the patient query to the appropriate specialized agent, while the Offline LAM & RAG Knowledge Pipeline provides the required clinical knowledge and patient context. The Large Action Model (LAM) receives information from the Vector Search & RAG Retriever Engine and the local patient context to support response generation by the specialized agent. All LAM inference and RAG processing are performed locally without external API dependency.
7. **Progress Tracking & Clinician Report Generation:** Recovery information and interaction outcomes are maintained within the local system. The Report Generation Agent within the Specialized Multi-Agent Execution Pool prepares structured patient progress information, which is made available through the Physician Escalation & Weekly Summary Portal for clinician follow-up.

6 Domain-Specific Design

Design Component	Details / Reference
Database Schema / ER Diagram	Local encrypted SQLite (SQLCipher) database with tables for: patient_profile, daily_checkins, interaction_logs, medication_schedule, pain_trend_history, triage_alerts, and report_summaries. Foreign key: patient_id.
ML Model Architecture	Large Action Model (LAM) architecture using a quantized local language model in GGUF Q4.K.M format as the underlying reasoning and action-generation engine. The model is executed locally through llama.cpp bindings (llama-cpp-python) and integrated with specialized agents and deterministic tools for postoperative care workflows. A fine-tuned BERT / Sentence-BERT embedding model is used for semantic intent matching and RAG retrieval.
Dataset Description	Knowledge base: Vetted orthopedic clinical protocols (discharge booklets, rehabilitation SOPs, DVT/PE red-flag checklists, physiotherapy phase guides). Patient data: Synthetic postoperative recovery profiles generated from aggregated clinical guidelines – no real PHI/PII collected.
API Design	Internal REST-style API between Flutter frontend and Python agent backend (localhost via HTTP). Endpoints: /auth, /checkin, /query, /report, /vitals. No external API calls during runtime – fully offline operation.
Security Architecture	All local data encrypted using SQLCipher AES-256. No cloud data transmission. Patient IDs are anonymized tokens. Camera frames processed on-device and discarded after analysis. Logs stored with patient-scoped encryption keys.
Deployment Architecture	Offline-first mobile application (Flutter/Dart). Python agent backend bundled as a local service on the mobile device or a companion host machine. GGUF model loaded in RAM via llama.cpp; ChromaDB index loaded from local storage.
Other Design Artifacts	rPPG vital-sign pipeline: OpenCV + MediaPipe for facial landmark / fingertip region-of-interest extraction. Photoplethysmographic (PPG) signal extraction from green-channel pixel variance over 30-second camera recording.

7 Algorithms / Methodology

Algorithm 1: Patient Query Scope Validation

This algorithm validates whether the patient's query falls within the supported scope of the postoperative chatbot before forwarding it to the AI processing module.

```
1 Algorithm 1: Patient Query Scope Validation
2 Input: Patient Query Q, Surgery Information S
3 Output: Validated Query or Referral Message
4
5 1: // Step 1: Retrieve Patient and Surgery Information
6 2: PatientContext = RetrievePatientContext(S)
7 3: SurgeryType = PatientContext.surgery_type
8 4: AnatomicalRegion = PatientContext.anatomical_region
9
10 5: // Step 2: Validate Query Against Supported Clinical Scope
11 6: QueryDomain = DetermineQueryDomain(Q)
12
13 7: if QueryDomain != "ORTHOPEDIC_POSTOPERATIVE_CARE" then
14 8:     ReferralMessage = GenerateOutOfScopeResponse(
15 9:         "The chatbot currently supports only orthopedic
16 postoperative queries."
17 10:     )
18 11:     RecommendHealthcareProfessional(ReferralMessage)
19 12:     return ReferralMessage
20 13: end if
21
22 14: // Step 3: Identify Anatomical Region
23 15: QueryRegion = IdentifyAnatomicalRegion(Q)
24
25 16: // Use the patient's surgery region if the query does not
26 17: // explicitly specify an anatomical region
27 18: if QueryRegion == NULL then
28 19:     QueryRegion = AnatomicalRegion
29 20: end if
30
31 21: // Step 4: Validate Supported Anatomical Region
32 22: SupportedRegions = { "HIP", "BELOW_HIP" }
33
34 23: if QueryRegion not in SupportedRegions then
```

```

34 24:      ReferralMessage = GenerateOutOfScopeResponse(
35 25:          "The requested anatomical region is currently not
        supported."
36 26:      )
37 27:      RecommendHealthcareProfessional(ReferralMessage)
38 28:      return ReferralMessage
39 29: end if
40
41 30: // Step 5: Forward Validated Query
42 31: ValidatedQuery = PrepareValidatedQuery(
43 32:     Q,
44 33:     PatientContext,
45 34:     QueryRegion
46 35: )
47 36: ForwardToAIProcessingModule(ValidatedQuery)
48
49 37: return ValidatedQuery

```

Listing 1: Patient Query Scope Validation Algorithm

Algorithm 2: Safety Triage and Multi-Intent Routing

This algorithm serves as the first-pass deterministic safety guard before any AI inference occurs. It classifies every patient query into an emergency risk level and routes it to the appropriate specialized agent.

```

1 Algorithm 1: Safety Triage and Multi-Intent Routing
2 Input:  Query string Q, Patient Context P,
3         Pain Log H_pain, Vitals V, Scan Inputs I_scan
4 Output: Safety Risk Level S, Intent Domain I, Target Agent A
5
6 1: RedFlags_Physical      = { "calf swelling", "calf warmth", "chest
    pain",
7                               "shortness of breath", "fever above
    101",
8                               "purulent discharge", "foul odor",
9                               "cannot move leg", "joint deformity" }
10 2: RedFlags_Psychological = { "suicidal ideation", "severe panic",
11                               "total hopelessness", "extreme
    distress" }
12 3: YellowFlags            = { "mild redness", "increasing pain trend
    ",

```



```
13         "skipped medication", "sleeplessness
14     ",
15         "mild nausea" }
16 4:  // Step 1: Deterministic Red-Flag Safety Verification
17 5:  for term in RedFlags_Physical UNION RedFlags_Psychological do
18 6:      if term is semantically present in Q or ImageAnalysis(I_scan
19     ) then
20 7:          S = "RED_EMERGENCY"
21 8:          A = EmergencyEscalationAgent
22 9:          LogUrgentPhysicianAlert(P.patient_id, Q, term)
23 10:         return S, "Emergency", A
24 11:     end if
25 12: end for
26
27 13: // Step 2: Intent Domain Classification (Embedding Similarity)
28 14: QueryVector      = GenerateEmbedding(Q)
29 15: SimilarityScores = CosineSimilarity(QueryVector,
30     IntentDomainEmbeddings)
31 16: I = argmax(SimilarityScores)
32
33 17: // Step 3: Yellow-Flag Warning Check
34 18: if Max(H_pain.recent_scores) >= 8 or
35     V.hrv_ms < Threshold_HRV or
36     Any(term in Q for term in YellowFlags) then
37 19:     S = "YELLOW_CONSULT"
38 20: else
39 21:     S = "GREEN_NORMAL"
40 22: end if
41
42 23: // Step 4: Map Intent to Target Agent
43 24: A = DomainAgentMap[I]
44 25: return S, I, A
```

Listing 2: Safety Triage and Multi-Intent Routing Algorithm

Algorithm 3: Dynamic Context RAG Retrieval & Guardrailed Response Generation

This algorithm governs how each specialized agent constructs a grounded, safety-constrained response using the offline RAG pipeline and local LAM inference.

```
1 Algorithm 2: Dynamic RAG Retrieval and Guardrailed Response
   Generation
2 Input: Target Agent A, Patient Context P,
3       User Query Q, Safety Risk Level S
4 Output: Grounded Clinical Response R
5
6 1: // Step 1: Build Contextualized Vector Retrieval Query
7 2: RetrievalQuery = BuildQuery(
8 3:     Procedure      = P.surgery_type,
9 4:     Timeline       = "Postop Day " + P.postop_day_count,
10 5:     QueryText      = Q
11 6: )
12 7: QueryEmbedding = GenerateEmbedding(RetrievalQuery)
13
14 8: // Step 2: Query ChromaDB Vector Knowledge Base
15 9: RelevantProtocols = ChromaDB.SimilaritySearch(
16 10:     query_vector    = QueryEmbedding,
17 11:     top_k           = 3,
18 12:     metadata_filter = { "category": A.domain_name }
19 13: )
20
21 14: // Step 3: Construct Grounded System Prompt
22 15: SystemPrompt = ConstructPrompt(
23 16:     Role              = A.SystemInstructions,
24 17:     PatientMetadata   = P,
25 18:     ClinicalProtocols = RelevantProtocols,
26 19:     SafetyConstraint   = S,
27 20:     UserQuery         = Q
28 21: )
29
30 22: // Step 4: On-Device Model Inference
31 23: RawResponse = LocalLAM.Generate(
32 24:     prompt          = SystemPrompt,
33 25:     temperature     = 0.2,           // Low temperature for medical
                                         consistency
34 26:     max_tokens      = 400
35 27: )
36
37 28: // Step 5: Clinical Guardrail Verification and Logging
38 29: R = ValidateAndAppendDisclaimers(RawResponse, S)
39 30: EncryptedSQLite.StoreInteraction(P.patient_id, Q, R, S,
```

```

    Timestamp() )
40
41 31: return R

```

Listing 3: Dynamic RAG Retrieval and Guardrailed Response Generation Algorithm

Algorithm 4: Periodic Recovery Risk Trend Aggregation

This algorithm runs on a scheduled daily/weekly basis to detect deteriorating recovery patterns and compile clinical summaries for physician review.

```

1 Algorithm 3: Recovery Risk Trend Aggregation
2 Input: Patient ID id, Time Window W (7 Days)
3 Output: Aggregated Risk Status, Weekly Summary Report
4
5 1: CheckinLogs = EncryptedSQLite.GetCheckins(id, W)
6 2: PainTrend = ComputeSlope(CheckinLogs.pain_scores)
7 3: AnxietyTrend = ComputeSlope(CheckinLogs.gad7_scores)
8 4: AdherenceRate = Count(CheckinLogs.completed_exercises) /
9     TotalPrescribed(W)
10
11 5: if PainTrend > 0.15 and AnxietyTrend > 0.20 then
12 6:     RiskLevel = "HIGH_RECOVERY_DISTRESS"
13 7: else if AdherenceRate < 0.50 then
14 8:     RiskLevel = "REHAB_NON_COMPLIANCE"
15 9: else
16 10:     RiskLevel = "NORMAL_PROGRESSION"
17 11: end if
18
19 12: Report = GenerateSummaryPDF(
20     id, PainTrend, AnxietyTrend,
21     AdherenceRate, RiskLevel
22 )
23 13: return RiskLevel, Report

```

Listing 4: Recovery Risk Trend Aggregation Algorithm

Algorithm 5: Medication Reminder and Confirmation

This algorithm monitors the patient's prescribed medication schedule, sends reminders at the scheduled time, records patient confirmation, and generates follow-up reminders

for unconfirmed medications.

```
1 Algorithm 5: Medication Reminder and Confirmation
2 Input: Patient ID id, Medication Schedule M
3 Output: Medication Reminder Status, Medication-Adherence Record
4
5 1: MedicationList = RetrieveMedicationSchedule(id)
6
7 2: for each Medication m in MedicationList do
8 3:     MedicineName = m.name
9 4:     Dosage        = m.dosage
10 5:     PrescribedTime = m.time
11
12 6:     // Step 1: Determine Next Scheduled Medication Time
13 7:     NextMedicationTime = GetNextScheduledTime(PrescribedTime)
14
15 8:     // Step 2: Send Medication Reminder
16 9:     WaitUntil(NextMedicationTime)
17 10:    SendReminder(
18         id,
19         MedicineName,
20         Dosage,
21         PrescribedTime
22     )
23
24 11:    // Step 3: Obtain Patient Confirmation
25 12:    Confirmation = GetPatientConfirmation(id)
26
27 13:    if Confirmation == "TAKEN" then
28 14:        MedicationStatus = "TAKEN"
29 15:        ConfirmationTime = Timestamp()
30 16:        StoreMedicationRecord(
31             id,
32             MedicineName,
33             Dosage,
34             PrescribedTime,
35             MedicationStatus,
36             ConfirmationTime
37         )
38
39 17:    else
40 18:        MedicationStatus = "UNCONFIRMED"
```

```
41 19:      StoreMedicationRecord(  
42         id,  
43         MedicineName,  
44         Dosage,  
45         PrescribedTime,  
46         MedicationStatus,  
47         NULL  
48     )  
49  
50 20:      // Step 4: Generate Follow-up Reminder  
51 21:      SendFollowUpReminder(  
52         id,  
53         MedicineName,  
54         Dosage  
55     )  
56 22:      end if  
57  
58 23:      // Step 5: Continue Monitoring Remaining Schedule  
59 24: end for  
60  
61 25: return MedicationStatus, MedicationAdherenceRecord
```

Listing 5: Medication Reminder and Confirmation Algorithm

8 Technology Stack

Component / Layer	Version	Technology / Tool & Justification
Mobile Frontend	Flutter 3.22.x (Dart 3.4.x)	High-performance, cross-platform (Android/iOS) UI with native camera access for rPPG vitals and scan uploads.
Large Action Model (LAM)	GGUF Q4_K_M	Local action-oriented model using a quantized language model executed through llama.cpp. The LAM interprets patient requests, selects appropriate agents and tools, and executes predefined postoperative care workflows without external API dependency.
Agent Framework	LangChain 0.2.x / CrewAI	Structured multi-agent role specification, task routing, and deterministic tool execution chains in local execution mode.
Vector Database (RAG)	ChromaDB 0.5.x / FAISS	Lightweight embedded vector database operating locally for semantic similarity search over vetted clinical protocol documents.
Local Database	SQLite 3.45.x (SQLCipher)	AES-256 encrypted on-device relational database for patient context, check-in logs, and pain trend history.
Vitals & Vision	OpenCV 4.9.x / MediaPipe	Optical camera processing for rPPG pulse/HRV extraction from fingertip video and wound photo analysis.
Programming Stack	Python 3.11.x / Dart 3.4.x	Python for AI/agent backend; Dart (Flutter) for mobile frontend development.
Version Control	Git 2.45+	GitHub-based repository management for collaborative development and version history.

Table 3: Project Technology Stack

9 Hardware and Software Requirements

Hardware Requirements

Minimum Development Workstation:

Component	Specification
Processor (CPU)	Intel Core i7 (12th Gen+) / AMD Ryzen 7 5800X / Apple M1 Pro Silicon or higher
System Memory (RAM)	16 GB DDR4/DDR5 minimum; 32 GB recommended for model quantization & local benchmarks
GPU	NVIDIA GeForce RTX 3060 (6 GB VRAM+) / Apple Unified Memory (16 GB+) for local GGUF model testing
Storage	512 GB NVMe M.2 SSD; minimum 25 GB dedicated for local model weights and vector store indices

Target Mobile Client Device:

Component	Specification
Operating System	Android 10.0 (API Level 29) or higher / iOS 14.0 or higher
Processor	Octa-core 2.0 GHz or higher (e.g., Snapdragon 778G / Apple A13 Bionic or better)
System Memory (RAM)	4 GB minimum; 6 GB+ recommended for smooth local quantized LAM execution
Available Storage	2.5 GB free internal storage (app bundle, GGUF weights, local vector index)
Camera Hardware	12 MP camera with autofocus and LED flash (wound photo captures and fingertip rPPG vitals)

Software Requirements

Software		Version / Details
Operating System		Windows 11 Pro 64-bit / macOS Sonoma 14.x / Ubuntu 22.04 LTS
Programming Language(s)	Lan-	Python 3.11.9 (AI/Agent Backend), Dart 3.4.x (Flutter Mobile Frontend)
Framework(s)		Flutter SDK 3.22.x, LangChain 0.2.x, CrewAI (Local Execution Mode)
Database		SQLite 3.45.x with SQLCipher AES-256 encryption; ChromaDB 0.5.x (Vector Store)
IDE / Editor		VS Code 1.90+, Android Studio Jellyfish (2023.3.1+)
Libraries / Packages		llama-cpp-python==0.2.77, langchain==0.2.5, chromadb==0.5.0, opencv-python==4.9.0, pydantic==2.7.4, numpy, pandas, sqlite3
Version Control		Git 2.45+, GitHub repository management
Other Dependencies		Node.js 20.x LTS (build toolchain), Android SDK (API 29+), Xcode 15+ (iOS build)

10 Design Considerations (Non-Functional)

Design Aspect	Description
Scalability	The modular multi-agent architecture allows new domain agents (e.g., physiotherapy video guidance, gait analysis) to be added independently without redesigning the orchestration layer. The ChromaDB vector store can be extended with additional knowledge base documents with no architectural change.
Performance	The quantized GGUF model (Q4_K_M) achieves inference latency under 2 seconds on mid-range mobile hardware. Embedded ChromaDB vector search returns top-k results in under 100ms. SQLite read/write operations are sub-millisecond for routine check-in logging.
Security	All patient data is stored locally in AES-256 SQLCipher encrypted SQLite. No data is transmitted to external servers during runtime. Patient identifiers are tokenized MRN hashes. Camera frames are processed in memory and never persisted to disk.
Reliability	The Safety Triage Engine uses deterministic rule-based logic for RED flag detection, ensuring reliable emergency escalation even if the local AI model fails or is unavailable. Fallback responses are pre-configured for all critical clinical scenarios.
Maintainability	Each agent module is implemented as an independent LangChain / CrewAI agent with its own system prompt and tool set. The orthopedic knowledge base can be updated by replacing vector embeddings without modifying application code.
Privacy	No Personal Health Information (PHI) or Personally Identifiable Information (PII) is collected, transmitted, or stored in identifiable form. All data is processed entirely on-device. The system is designed to comply with health data confidentiality norms.
Ethical Considerations	The system explicitly appends medical disclaimers to all AI-generated responses, reminding patients to verify guidance with their treating physician. The AI is designed as a support tool, not a diagnostic authority. Emergency escalation always overrides AI response generation.

11 Expected Outcome

The successful completion of this project is expected to deliver the following outcomes:

1. **Functional Multi-Agent AI Application:** A working mobile application featuring 11 specialized AI agents capable of providing personalized, grounded postoperative guidance for below-hip orthopedic patients across domains including rehabilitation, wound care, medication adherence, daily activities, nutrition, and mental wellbeing.
2. **Privacy-Preserving Offline Deployment:** A fully offline-capable system with no cloud dependency, ensuring 100% on-device processing of sensitive patient recovery data through quantized local LAM inference and embedded vector search.
3. **Clinically Safe Triage System:** A deterministic Emergency & Safety Triage Engine that reliably classifies patient queries into RED (emergency), YELLOW (consult), and GREEN (routine) risk levels, preventing unsafe AI responses in critical medical scenarios.
4. **Automated Clinician Reporting:** An automated weekly recovery summary pipeline that aggregates patient pain trends, exercise compliance, and psychometric scores into structured PDF reports for reviewing orthopedic surgeons and physiotherapists.
5. **Validated Knowledge Base:** A ChromaDB vector knowledge base populated with vetted orthopedic postoperative protocols, discharge guidelines, and phase-wise physiotherapy plans across below-hip procedure types (THA, TKA, fracture fixation, ankle reconstruction).
6. **Research Contribution:** A publishable research contribution demonstrating the viability of offline multi-agent AI architectures for privacy-sensitive clinical support applications, addressing identified gaps in existing postoperative care AI systems.

12 References

1. J. Li, Y. Zhang, Z. Zhang, Y. Zhou, Y. Gao, X. Li, and S. Fan, "A randomized controlled trial of a WeChat-based artificial intelligence agent for postoperative care in orthopedic patients," *npj Digital Medicine*, vol. 9, Art. no. 105, 2026. doi: 10.1038/s41746-025-02269-8.

2. H. Chen, Y. Hou, M. Zhong, et al., “Meissa: Multi-modal medical agentic intelligence,” *arXiv preprint arXiv:2603.09018*, 2026. Available: <https://github.com/Schutture/Meissa>.
3. M. Hosseini and H. Seilani, “Agentic AI system for stress monitoring,” *International Journal of Computational Intelligence Systems*, vol. 19, Art. no. 95, 2026. doi: 10.1007/s44196-026-01215-0.
4. J. Wang, Z. Xu, X. Zhao, et al., “Large action models: From inception to implementation,” *arXiv preprint arXiv:2412.10047*, 2024.
5. Y. Zhu, Z. He, H. Hu, X. Zheng, X. Zhang, Z. Wang, J. Gao, L. Ma, and L. Yu, “MedAgentBoard: Benchmarking multi-agent collaboration with conventional methods for diverse medical tasks,” *arXiv preprint*, 2025.

Faculty Remarks

Guide Remarks

Project Guide

Name: Ms. Gopika Muraleedas

Signature: _____



Project Coordinator

Name: _____

Signature: _____

A Instructions to Students

General Guidelines

- Complete all sections of the report before submission.
- Present technical content in a clear, concise, and professional manner.
- Use original work supported by appropriate references and citations.
- AI-assisted tools may be used only as supporting aids. Students are responsible for ensuring the originality, technical accuracy, and completeness of all submitted content.
- Reports containing unverified, generic, or AI-generated content without substantial student contribution may be rejected.
- All figures, tables, and diagrams must be clearly labeled and referenced in the text.
- Remove all placeholder text before final submission.

Section-wise Guidelines

Project Title Provide a clear and meaningful title that reflects the proposed work.

SDG Mapping Identify the primary SDG and justify its relevance to the proposed project.

Project Timeline Summarize the major milestones, deliverables, and expected completion schedule.

System Architecture

Present the overall system architecture with a brief description of the major components.

Module Description

Describe the functionality of each module and its role in the system.

System Workflow

Illustrate the operational workflow using an appropriate diagram.

Domain-Specific Design

Include only the design artifacts relevant to the project domain.

Technology Stack

List the programming languages, frameworks, tools, libraries, and platforms used.

Requirements

Specify the hardware and software required for development and deployment.

Design Considerations

Discuss relevant aspects such as performance, scalability, security, maintainability, and reliability.

Expected Outcome

Summarize the expected deliverables and anticipated project outcomes.

References

Cite all referenced literature using IEEE style.

Submission Checklist

- All required sections are completed.
- Figures, tables, and references are properly numbered and cited.
- Grammar and formatting have been reviewed.
- The report has been verified and approved by the project guide prior to submission.